

Open-Source Reimplementation of AlphaRouter for Quantum Circuit Routing

Report for the TU Delft EEMCS Honours Programme

Jakub Pietrzak[1]

Supervisor: Sebastian Feld[2]

[1]**EEMCS**
Delft University of Technology
Delft, The Netherlands

[2]**Quantum & Computer Engineering**
QuTech
Delft University of Technology
Delft, The Netherlands

Abstract

Quantum circuit routing is a compilation problem in which SWAP operations are inserted so that two-qubit gates respect the limited connectivity of a target quantum device. Recent work by Tang et al. proposed AlphaRouter, a reinforcement-learning-based router trained with Monte Carlo tree search, and reported lower routing overhead than heuristic baselines such as SABRE. This report presents an open-source reimplementation of AlphaRouter. The implementation is evaluated on line-topology routing tasks with trivial initial mappings, including single-gate instances with 8 and 10 qubits and multi-gate instances on a 4-qubit device with circuits of up to 10 gates. The reimplementation routes the 8-qubit single-gate setting close to the optimal SWAP count, but its performance degrades on the 10-qubit single-gate setting and remains worse than SABRE on the multi-gate setting. Curriculum learning improves the trainability of the multi-gate agent in the tested setting, increasing the maximum efficiently routed circuit length from 5 to 10 gates, but the best curriculum-trained agent still requires approximately 50% more SWAPs than SABRE. These results do not replicate the performance reported by Tang et al.; instead, they show that the reproduced method is sensitive to implementation choices, hyperparameters, training schedule, and available compute. The released implementation provides a basis for further investigation of learning-based quantum circuit routing, but SABRE remains the stronger and more reliable method under the experiments reported here.

1 Introduction

Quantum computers offer quadratic, and in some cases exponential speedups over the best- known classical algorithms for several important computational problems [1, 2, 3, 4]. A quantum program is commonly represented as a circuit: an ordered sequence of one- and two-qubit gates acting on quantum bits [5, 6]. To execute such a circuit on a real device, a compiler must translate it to the target backend's native operations while respecting hardware connectivity constraints [7, 8].

In many near-term devices, two-qubit gates can be applied directly only to selected pairs of physical qubits, so a logical two-qubit gate between non-adjacent qubits generally requires additional compilation steps [6, 7, 9]. Qubit mapping assigns logical qubits in the input circuit to physical qubits in the hardware coupling graph, while qubit routing updates this assignment during execution by inserting SWAP operations or equivalent permutations until the required two-qubit gates are adjacent [8, 9, 10, 11].

Routing is a complex combinatorial problem known to be NP-Hard [8, 10, 12]. This difficulty matters operationally because each inserted SWAP decomposes into hardware-level gates, increasing gate count, circuit depth, and usually the probability of failure on noisy hardware [9, 13]. The effect is particularly important in the noisy intermediate-scale quantum (NISQ) regime, where devices are not yet fully fault tolerant and computation is limited by noise and finite coherence times [14]. Practical compilers therefore rely heavily on heuristic routing methods rather than exact global optimization for large circuits [6, 9, 12]. SABRE is a representative example: it uses bidirectional heuristic search and a decay mechanism to trade off SWAP count and circuit depth on arbitrary coupling graphs [9]. Such rule-based routers can be effective, but their scoring functions and search neighborhoods encode fixed design choices that may need to be redesigned when the target topology, calibration data, or optimization objective changes [9, 13, 15].

Reinforcement learning (RL) has therefore been investigated as a way to learn quantum-compilation policies, including initial mapping, routing, and broader circuit-optimization tasks [8, 16, 17, 18, 19]. In an RL formulation of routing, an agent observes a representation of the circuit and hardware state and selects actions, SWAP gate insertions, to satisfy connectivity constraints while minimizing routing overhead [8, 11, 15, 16]. Tang et al. proposed AlphaRouter, an RL-based router that combines learned decision-making with Monte Carlo tree search and reports up to 20% lower routing overhead than state-of-the-art heuristic baselines [15]. This makes AlphaRouter a relevant case study for assessing whether learning-based routing can improve over hand-designed compilation heuristics.

Learning-based compilers also introduce reproducibility risks beyond those of deterministic heuristics, because results depend on implementation details, environment dynamics, random seeds, hyperparameters, benchmark generation, and evaluation protocols [8, 20, 21, 22]. For computational research, reproducibility depends on access to the code, data, software environment, and execution instructions, not only on the textual description of the method [20, 21]. Even when code is available, reuse is not guaranteed: Trisovic et al. found that 74% of analyzed R files from public replication datasets failed to execute without error initially, and 56% still failed after automatic code cleaning [23]. An independent and documented implementation of AlphaRouter is therefore useful for checking the original claims and for providing a baseline that future RL-based routing work can inspect, reuse, and extend [15, 20].

This report presents an independent reimplementation of AlphaRouter for qubit routing and releases the implementation to support inspection, reuse, and extension. We compare the reimplementation with the original paper's reported claims under the experimental conditions available to us and investigate training techniques intended to improve the routing agent. The main contributions are: (i) a documented implementation of the AlphaRouter architecture, including the RL agent and tree-search components; (ii) an empirical comparison with the routing behavior reported by Tang et al.; and (iii) an evaluation of additional training techniques for improving the agent.

2 Background

2.1 Routing and Reinforcement Learning

Quantum circuit routing is a compilation subproblem that modifies a logical circuit so that every two-qubit gate can be executed on a target device with restricted qubit connectivity [9, 10, 11]. The hardware is commonly represented as a coupling graph $G = (V, E)$, where vertices are physical qubits and edges indicate pairs of qubits on which a two-qubit operation can be applied directly [9, 10]. A qubit mapping is an injective map $\mu : Q_L \to V$ from logical qubits to physical qubits. A two-qubit gate acting on logical qubits (q_i, q_j) is executable under μ when $\{\mu(q_i), \mu(q_j)\} \in E$; otherwise, a router must change the mapping before executing the gate. In SWAP-based routing, this is done by inserting a SWAP on a hardware edge $\{u, v\} \in E$, which exchanges the logical qubits currently assigned to u and v [9, 11, 15].

The objective of routing is usually to minimize the overhead introduced by these additional operations, commonly measured by the number of inserted SWAPs, the increase in circuit depth, or the number of additional two-qubit gates after decomposition [9, 10, 11]. This objective is non-local: a SWAP that makes the next gate executable can make later gates harder to route, so greedy local decisions need not be globally optimal. Formal versions of qubit allocation and routing-overhead minimization are computationally hard, which motivates the use of heuristic and learning-based methods for non-trivial circuit sizes [10, 12, 15].

Routing can be formulated as a Markov decision process (MDP), the standard mathematical model for many reinforcement learning problems [11, 15, 16, 24]. An MDP is a tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' \mid s, a)$ is the transition distribution, $R(s, a, s')$ is the reward, and $\gamma \in [0, 1]$ discounts future rewards [24]. In a routing MDP, a state can contain the remaining unscheduled circuit, the current logical-to-physical mapping, and the hardware coupling graph; an action is typically the choice of a SWAP on one of the hardware edges; and the transition updates the mapping and removes gates that have become executable [11, 15, 16]. The reward is chosen to encode routing quality, for example by rewarding newly scheduled gates and penalizing inserted SWAPs [15]. A policy $\pi_\theta(a \mid s)$ is then trained to maximize the expected discounted return

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right], \tag{1}$$

where $\tau = (s_0, a_0, r_0, \ldots, s_T)$ is a trajectory generated by repeatedly interacting with the routing environment [24].

This formulation is useful because it separates the routing problem into an environment, which defines legal SWAPs and state transitions, and an agent, which learns a decision policy from rewards.

2.2 qgym

OpenAI Gym introduced a common interface for reinforcement learning environments, allowing agents to interact with tasks through a standardized loop based on environment reset, action selection, state transition, and reward observation [25]. qgym adapts this idea to quantum compilation by providing Gym-compatible environments for training and benchmarking RL-based quantum compilers [8]. The framework is designed to abstract away compilation details that are not central to the learning algorithm while still exposing the decision structure needed by an RL agent [8].

qgym is relevant here because AlphaRouter is a reinforcement learning agent, it depends on a precise environment definition for states, legal actions, rewards, and terminal conditions. The qgym framework provides three compilation-oriented environments: initial mapping, routing, and scheduling [8]. In the initial-mapping environment, the agent chooses an assignment from logical to physical qubits before routing starts. In the routing environment, the agent inserts SWAPs to satisfy the connectivity constraints of the target topology. In the scheduling environment, the agent decides how to order executable operations under hardware and dependency constraints [8]. This separation is useful because it makes the routing task narrower and easier to benchmark, but it also means that results for routing alone should not be confused with full end-to-end quantum compilation.

2.3 AlphaGo, AlphaZero, and Monte Carlo Tree Search

AlphaGo demonstrated the effectiveness of combining neural networks with tree search in a large sequential decision problem [26]. AlphaGo used policy networks to guide move selection, value networks to evaluate board positions, and Monte Carlo tree search (MCTS) to improve decisions at search time [26]. AlphaZero later generalized this pattern by learning from self-play using only the rules of the game, replacing handcrafted evaluation functions and domain-specific search enhancements with a learned policy-value network and MCTS [27].

MCTS is a decision-time planning algorithm that incrementally builds a search tree over states and actions [28]. A standard MCTS iteration consists of selection, expansion, simulation or value evaluation, and backpropagation of the resulting value estimate through the visited nodes [28]. The Upper Confidence Bounds applied to Trees (UCT) variant selects actions by balancing exploitation of high-value actions and exploration of actions with fewer visits [29]. In abstract form, a common UCT-style score for a child node i of parent p is

$$\mathrm{UCT}(i) = \bar{X}_i + c\sqrt{\frac{\ln N_p}{N_i}}, \tag{2}$$

where $\bar{X}_i$ is the empirical value estimate, N_i is the visit count of the child, N_p is the visit count of the parent, and c controls the exploration–exploitation trade-off [28, 29].

The analogy to routing is not exact. Go, chess, and shogi are adversarial two-player games, while qubit routing is a single-agent combinatorial optimization problem. The transferable idea is the algorithmic pattern: use a neural

model to bias or evaluate a search process, and use the improved decisions produced by search as training targets for the neural model. AlphaRouter adopts this pattern by using MCTS during training to generate stronger action and value targets for a learned routing agent [15].

2.4 Transformers

The Transformer is a neural sequence model based on attention mechanisms rather than recurrence or convolution [30]. Its central operation is scaled dot-product attention,

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \qquad (3)$$

where Q, K, and V are query, key, and value matrices and d_k is the key dimension [30]. Multi-head attention applies this operation in several learned representation subspaces and concatenates the resulting features, allowing the model to attend to different relations within the same input sequence [30]. Because the Transformer has no recurrent ordering bias, positional encodings are commonly added to the input embeddings so that the model can distinguish the positions of tokens in a sequence [30].

Transformers are a plausible architecture for circuit routing because a circuit is an ordered and dependency-constrained object rather than an unordered set of gates. A routing decision can depend on interactions between the current front layer, nearby future gates, the current mapping, and the topology of the target device. AlphaRouter uses a transformer-based actor-critic network to process an encoding of the remaining logical circuit and current qubit mapping, then output SWAP-action predictions and a state-value estimate [15]. In the original AlphaRouter design, each gate in the lookahead window is embedded using information about its mapped physical qubit targets and topological depth, and positional encodings are used because gate order carries physical meaning for routing [15].

The main advantage of this design is expressivity: self-attention allows each encoded gate to interact with other gates in the lookahead window.

2.5 AlphaRouter

AlphaRouter is an RL-based qubit router that combines a transformer policy-value network with MCTS during training [15]. It targets the routing problem for a fixed quantum-computer topology and a given initial mapping, rather than solving initial placement and routing jointly [15]. In the AlphaRouter formulation, the state consists of the remaining logical circuit and the current qubit mapping, the action space consists of SWAPs on hardware edges, and the goal is to find a short sequence of actions that routes the circuit until all gates are scheduled [15].

During training, AlphaRouter first generates routing experience by letting the transformer agent interact with the routing environment [15]. A replay buffer stores visited states, and MCTS is then applied to sampled states to produce improved action choices and value estimates [15]. The transformer is updated using a loss that combines a cross-entropy term between the network's action prediction and the MCTS-improved action target with a squared-error term between the network's state-value prediction and the MCTS value estimate [15]. This is structurally similar to AlphaZero-style training, but the domain is a single-agent routing task rather than a two-player game [15, 27].

At inference time, the original AlphaRouter uses the trained neural agent without MCTS in order to reduce compilation latency [15]. The agent repeatedly encodes the current state, predicts a SWAP action, applies the SWAP, greedily schedules newly executable gates, and stops once the full logical circuit has been routed [15]. The paper reports that this approach can reduce routing overhead relative to state-of-the-art heuristic baselines, including SABRE, with up to 20% lower routing overhead in the reported experiments [15].

2.6 Curriculum Learning

Curriculum learning trains a model on examples or tasks arranged from easier to harder, rather than sampling immediately from the full target distribution [31]. Bengio et al. motivated this approach as a way to improve optimization and generalization in non-convex learning problems, where the order of training examples can influence the solution found by the training process [31]. In reinforcement learning, curriculum learning can sequence tasks, start states, goals, environment parameters, or reward structures so that experience from easier tasks transfers to harder tasks [32].

Routing has natural curriculum variables. A curriculum can increase the number of logical qubits, circuit depth, number of two-qubit gates, topology diameter, irregularity of the coupling graph, distance between the qubits affected by the gates of the circuit, or difficulty of the initial mapping. It can also move from single-gate or short-circuit routing instances to multi-gate circuits with longer-range dependencies. Such a curriculum is plausible because early training on long circuits may provide sparse or noisy reward signals, while shorter circuits can expose the agent to successful terminal trajectories more frequently [32].

In this report, curriculum learning should be evaluated as a training technique for AlphaRouter, not as a change to the routing objective itself.

3 Implementation

This section describes the implementation used in this report. The codebase uses Python 3.12 and is organized around a qgym routing environment, a transformer-based policy/value network, a Monte Carlo tree search (MCTS) critic, a replay buffer of routing states, and an optional curriculum learning scheduler. The source code is available under https://github.com/GeneralKenobiJP/AlphaRouter

3.1 Software architecture

The central component is the `AlphaRouterAgent`, implemented as a PyTorch `nn.Module`. It owns the qgym routing environments, the trainable qubit-embedding table, the transformer policy/value network, the replay buffer, the MCTS object, and the optimizer.

The transformer outputs SWAP-action logits, with one action per edge in the hardware connection graph. The agent

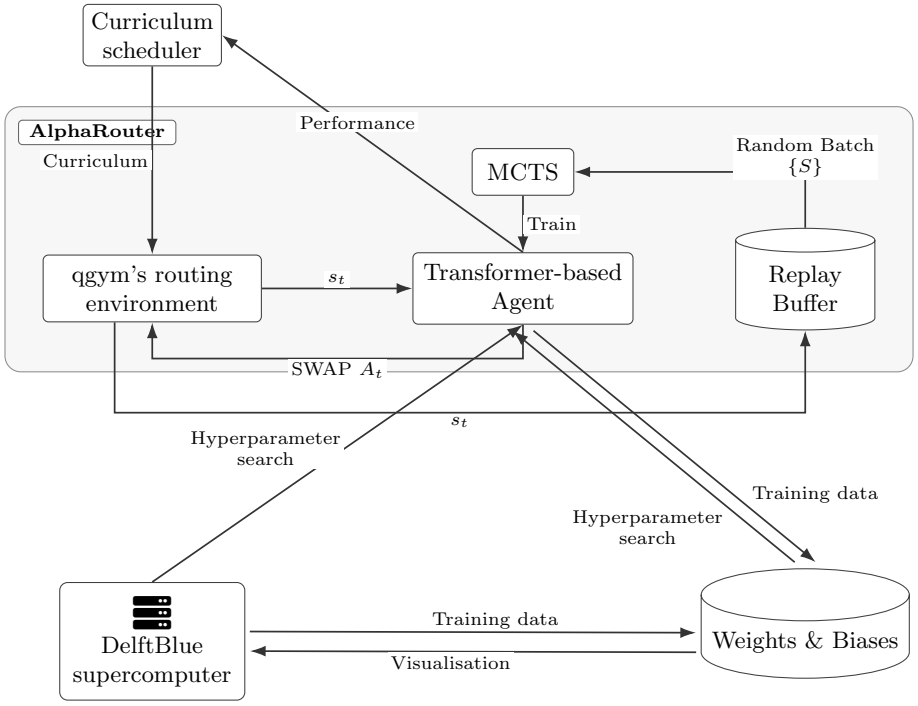


Figure 1: Architecture of the implementation: building blocks of the AlphaRouter, and the training enhancing methods: curriculum scheduler for curriculum learning, DelftBlue supercomputer for High-Performance Computing (HPC) hyperparameter search, Weights & Biases for visualisation and local hyperparameter search

chooses stochastically actions from the logits. Gates that are already executable are advanced automatically. The training is structured as an actor-critic architecture, with MCTS acting as the critic.

For the training we sample states from the Replay Buffer. To fill the replay buffer, we maintain a vectorized set of environments that try to route randomly generated circuits with the current weights of the transformer, thus producing new states.

The circuit generation algorithm can be freely modified by extending the `InteractionGenerator` class.

3.2 State encoding

The neural input is constructed as a learned feature matrix from the current `RoutingState`. For each interaction in the truncated interaction circuit, the implementation reads the two logical qubits, maps them to physical qubits using the current logical-to-physical mapping, retrieves trainable embeddings for the corresponding physical qubits, averages the two embeddings, appends a scalar dependency-depth feature, and pads the result if necessary. The dependency-depth feature is computed as one plus the number of previous gates sharing either of the two logical qubits. If fewer than `lookahead_length` gates are present, zero vectors are appended until the feature matrix has fixed length.

Formally, for a gate $g_t = (q_i, q_j)$, let $m(q)$ be the current physical qubit assigned to logical qubit q, and let E_p be the learned embedding of physical qubit p. The implemented gate feature is

$$x_t = \left[\frac{E_{m(q_i)} + E_{m(q_j)}}{2};\ d_t \right],$$

where d_t is the dependency-depth scalar computed by the implementation. The resulting matrix has shape

$$L \times (d_e + 1 + d_{\text{pad}}),$$

where L is the lookahead length, d_e is the physical-qubit embedding dimension, and d_{pad} is the optional padding dimension.

3.3 Transformer policy and value network

The neural network is implemented in the `Transformer` class, also a PyTorch `nn.Module`. Its input is first projected from the gate-feature dimension into a model dimension `d_model`. Sinusoidal positional encodings are added to the projected sequence. The sequence is then processed by a configurable stack of transformer encoder layers. Each layer consists of PyTorch multi-head self-attention, a residual connection with layer normalization, a two-layer feed-forward network with ReLU activation, and a second residual normalization block.

The network has two output heads. The policy head decodes each sequence position into SWAP-action logits and then averages these logits over the lookahead dimension to obtain one logit vector over the available SWAP actions. The value head maps each sequence position to a scalar and

averages the scalar values across the lookahead dimension to produce a single state value. Action selection is implemented by applying `log_softmax` to the logits and sampling from thus created distribution.

3.4 Monte Carlo tree search

A tree node stores a qgym `RoutingState`, a visit count, an accumulated value, and a dictionary from child nodes to the action used to reach them. States are memoized through a transposition table.

The implemented selection rule is value-UCB:

$$\mathrm{UCB}(s_t \to s_{t+1}) = \frac{Q(s_{t+1})}{N(s_{t+1}) + \epsilon} + c\sqrt{\frac{\log(N(s_t) + 1 + \epsilon}{N(s_{t+1}) + \epsilon}},$$

where s_{t+1} is a child of parent s_t, $Q(s)$ is the accumulated node value, $N(s)$ is the visit count, c is the exploration constant, and ϵ is an arbitrary small value to ensure safe mathematical operations. The implementation selects the first unused action index in the available SWAP-action range. Leaf evaluation is performed by embedding the leaf state and applying the transformer's value head. Terminal rewards are computed by applying -1 penalty per each SWAP insertion, and $+1$ reward per each gate surpass, which is equivalent to AlphaRouter's formulation of:

$$r_t = |G_t| - |G_t + 1| - 1,$$

where r_t is the reward at timestep t, and G_i is the remaining logical circuit.

After a fixed number of rollouts, the chosen MCTS action is the root child with maximum average value. This is contrary to the AlphaGo policy, but we have found the choice based on value to produce better results for this problem.

3.5 Training

To train the agent we sample states stored previously in the Replay Buffer. The training is enabled only after an adjustable threshold of states stored in the buffer is reached, or if all environments have finished and the buffer contains more states than the requested batch size. The transformer is trained using a policy-imitation term and a value-regression term:

$$\ell_\pi(s) = -\log \pi_\theta(a_{\mathrm{MCTS}} \mid s),$$
$$\ell_V(s) = \left(V_\theta(s) - \hat{V}_{\mathrm{MCTS}}(s)\right)^2.$$

Then, we compute a dynamic scaling coefficient

$$\lambda = \sqrt{\frac{\sum \ell_\pi^2}{\sum \ell_V^2}},$$

and compute the final combined loss function:

$$\ell(s) = \frac{\ell_\pi(s) + \lambda \ell_V(s)}{|\mathcal{A}|},$$

where $|\mathcal{A}|$ is the size of the action space. Gradients are backpropagated through the transformer and qubit-embedding parameters, clipped to norm 1, and updated with Adam optimizer. The formulation of the loss functions is equivalent to the one provided by Tang et al.

One divergence from the original paper is the learning rate: we use a fixed learning rate throughout the entire training process, whereas Tang et al. use a decaying learning rate[1]. It is a common practice to implement a dynamic learning rate with initial warmup stage and eventual decay stage while working with Transformers [30]. Therefore, it would be interesting to investigate that in the future.

3.6 Curriculum learning

The curriculum mechanism introduces dynamic changes of the interaction generator used by the qgym environments. `CurriculumScheduler` maintains a current level, a maximum level, the level increment size, proceed threshold, retreat threshold, and cooldown. Its default generator is `GateLengthCurriculumGenerator`, which returns qgym's `BasicInteractionGenerator(max_length=level)`. In other words, in the current implementation, curriculum difficulty is controlled through the maximum generated interaction-circuit length, but the implementation is flexible and adjustable to custom scheduling algorithms.

The scheduler evaluates progress using the ratio between the agent's mean SWAP count and the SABRE baseline mean SWAP count. After the cooldown period, the curriculum level is increased when performance is below or equal to the proceed threshold, and decreased when performance is above or equal to the retreat threshold. Upon each level change, the scheduler waits again for the duration of the cooldown before another change of level is possible to make the process more stable. When the curriculum updates, the agent rebuilds the vectorized training and test environments using the new generator and recomputes the SABRE benchmark for the new test circuits.

3.7 Hyperparameter search: HPC & Weights& Biases

Hyperparameter search is supported through two W&B runner scripts. The online runner defines a Bayesian sweep over the hyperparameter space, which is then managed entirely through the W&B API. The offline runner randomly samples the hyperparameter space locally, and stores run logs as W&B artifacts, which can be synchronized afterwards with the W&B servers to provide visualisation options. The offline script is meant to be run on a supercomputer, where internal nodes lack access to the internet, as is the case for the DelftBlue supercomputer, which was used for this research [33]. We provide scripts that manage hyperparameter search on DelftBlue out-of-the-box.

4 Experiments

4.1 Setup

We aimed to see if the performance of the AlphaRouter as claimed by Tang et al. could be replicated. Moreover, we wanted to answer the following questions:

[1]The reason was research time constraints of the Honours Programme.

- Does the reimplemented AlphaRouter solve single-gate routing instances reliably?
- Does it reduce routing overhead on multi-gate circuits compared with heuristic baselines?
- Does curriculum learning improve the agent's ability to learn?

Experiments included direct comparison with SABRE on the same circuits. Because we did not have access to an official implementation associated with the original publication, direct comparison between the two implementations on exactly the same instances was not possible. Instead, we compare our results with the reported aggregate results of the random-circuit benchmark.

The multiple-gate routing experiment has been performed on DelftBlue supercomputer [33]. Otherwise, experiments were run locally.

All experiments considered quantum hardware with line topology. A trivial initial mapping was used where i-th logical qubit corresponds to the i-th physical qubit.

Figure 2 shows the training results of 36 agents with different combinations of hyperparameters. Since most of the hyperparameter combinations do not achieve success, all agents have been trained as a part of the hyperparameter search. While examining plots further in this section, the reader should assume that only the most successful agent is analyzed, and that he was trained along with 35 different agents with different hyperparameter settings.

Curriculum performance thresholds were treated as hyperparameters for the sake of hyperparameter search. The cooldown period was fixed to be 20, the level increment size was fixed to 1, the maximum level was fixed to 10.

4.2 Single-gate routing

We present the results we obtained while routing just 1 gate between the two furthest qubits in the line topology.

Figure 3 shows that the agent was successful in the 8-qubit setting.

Figure 4 shows that for 10 qubits the agent managed to find a general direction but failed to learn the full logic, and thus did not match the baseline performance.

4.3 Multiple-gate routing

Here, we present the results when a maximum of 10 gates are to be routed.

Figure 5 shows that the agent failed to achieve good performance when compared with the baseline. Consideration of circuit length in Figure 6 suggests that the agent's routing performance degrades rapidly once the routed circuit length exceeds 5 gates.

The curriculum learning was then applied to try to help the agent with the training process. The reasoning was that if the agent failed to learn how to properly route up to 10 gates from scratch, but was successful whenever a small problem instance was generated, then perhaps a more successful training approach would be to start with small instances, and then gradually increase their size as the agent is able to demonstrate the capability to solve problems with the current difficulty.

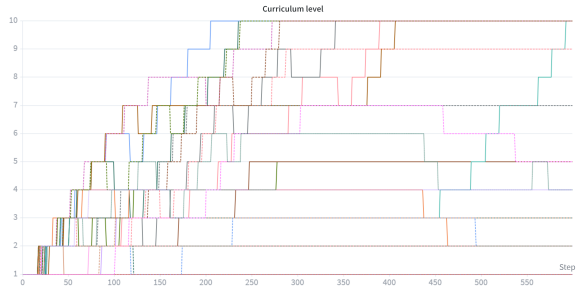


Figure 2: A summary of hyperparameter search of 36 runs for the multiple-gate 4-qubit setup - curriculum level per episode for each run. There is a big variance in performance - only a few of the hyperparameter configurations are successful.

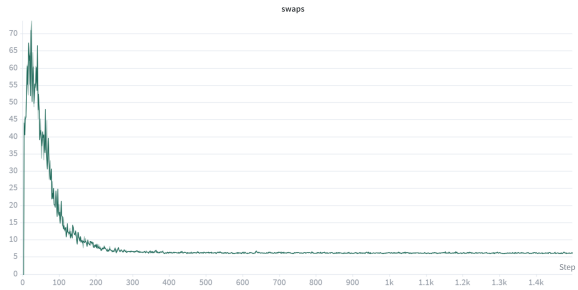


Figure 3: Agent training - Swaps per episode - Single gate, 8 qubits. The mean achieved performance is 6.0625 swaps, which is in line with the optimal number of 6 swaps for the instance.

In this tested configuration, curriculum learning significantly improved the agent's ability to learn, as shown in Figure 7. Thus trained agent was able to route up to 10 gates, but needed on average 50% more swaps than SABRE.

5 Conclusions & Future Work

This report presented an open-source Python 3.12 reimplementation of AlphaRouter for SWAP-based quantum circuit routing. Under the experimental conditions considered here, the reimplementation did not reproduce the

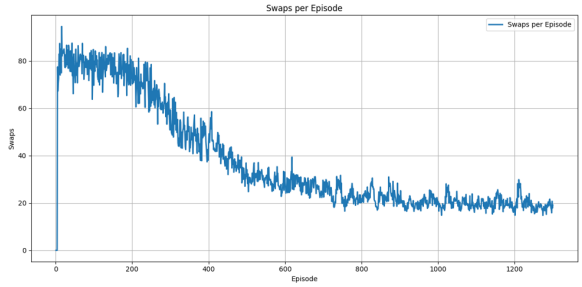


Figure 4: Agent training - Single gate, 10 qubits. Swaps per episode. The mean achieved performance is 19.1 swaps, which is considerably above the optimal number of 8 swaps for the instance, but is significantly better than a random walk.

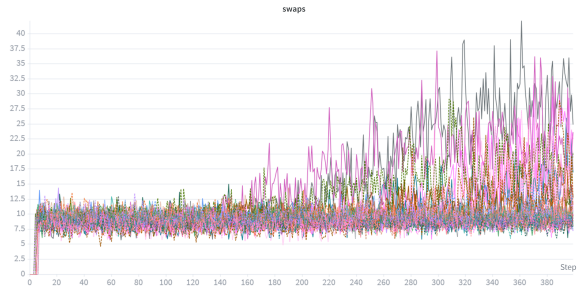


Figure 5: Swaps per episode for 36 agents with different hyperparameters (hyperparameter search result). None of the agents managed to reach below the mean of 4.5 at any point or below 6 at the end of the training, whereas SABRE reported the mean consistently between 2.1 and 2.8 swaps.

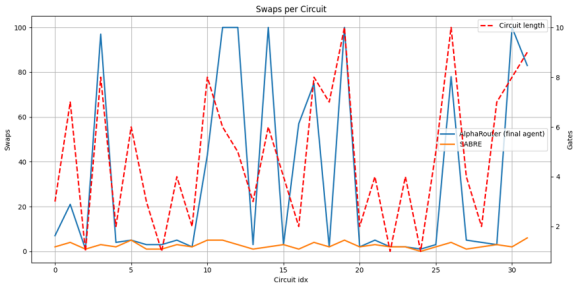
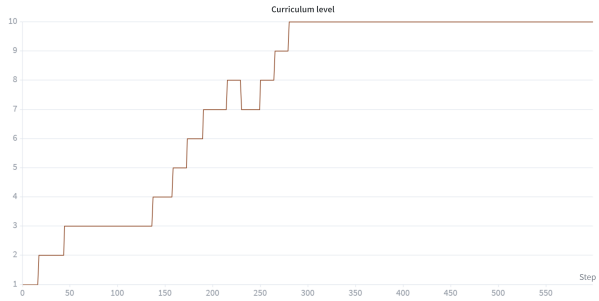


Figure 6: Swaps & circuit length per test circuit in the benchmark suite, comparison between an AlphaRouter agent trained without curriculum learning and the baseline SABRE. Performance diminishes rapidly once the circuit length exceeds 5 gates.



(a) Swaps per episode



(b) Curriculum level per episode

Figure 7: Agent training - Multiple gates, 4 qubits, curriculum learning. The trained agent needed on average 50% swaps more than SABRE.

performance reported by Tang et al. The strongest result was obtained on the single-gate 8-qubit line-topology task, where the agent reached an average of 6.0625 SWAPs, close to the optimal value of 6 SWAPs for the tested instance. Performance was weaker on the 10-qubit single-gate task, where the agent learned a useful direction but did not achieve optimal performance.

For multi-gate routing on a 4-qubit line topology with circuits of up to 10 gates, the best curriculum-trained agent required approximately 50% more SWAPs than SABRE. Without curriculum learning, the agent did not learn to route the 10-gate setting reliably; with curriculum learning, it reached the 10-gate setting after first training on shorter instances. This suggests curriculum learning is a better training framework than a standard approach for this problem.

Furthermore, the experiments show that extensive hyperparameter search is essential to finding performant deep RL-based routing agents.

The main negative result is clear: in this implementation and evaluation setting, SABRE remains substantially more reliable and more efficient than the reproduced AlphaRouter agent. We have not managed to replicate the findings of Tang et al. (for comparison, see: fig 8), but we acknowledge that, despite our best effort to closely follow the original paper, there may be differences in implementation or available compute power that have impact on the agent's performance. One difference that we are aware of is the learning rate. How impactful of a difference it is remains to be researched.

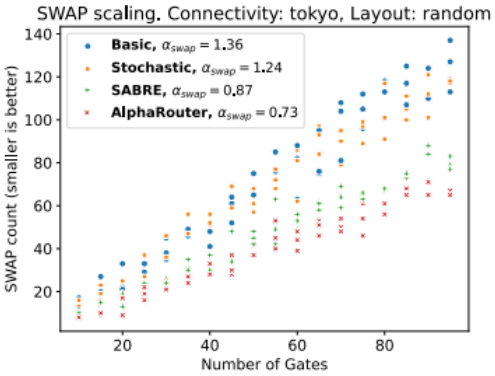


Figure 8: AlphaRouter performance reported by Tang et al. [15]

The major contribution of this work is that it provides an implementation that can be freely further inspected, experimented with, and extended by other researchers.

One direction of possible further research would be to implement a more transformer-like varying learning rate, conduct hyperparameter search, and benchmark such an agent to see if it matches the findings of Tang et al. more closely. A second direction is to extend curriculum learning beyond circuit length by varying topology size, initial mapping difficulty, and gate-distance distribution. Finally, the implementation should be benchmarked on non-line topologies and larger circuits before making any claim about scalability or practical compiler performance.

References

[1] A. Montanaro, "Quantum algorithms: an overview," *npj Quantum Information*, vol. 2, p. 15023, 2016. [Online]. Available: https://doi.org/10.1038/npjqi.2015.23

[2] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997. [Online]. Available: https://doi.org/10.1137/S0097539795293172

[3] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, ser. STOC '96. New York, NY, USA: Association for Computing Machinery, 1996, p. 212–219. [Online]. Available: https://doi.org/10.1145/237814.237866

[4] A. M. Childs, R. Cleve, E. Deotto, E. Farhi, S. Gutmann, and D. A. Spielman, "Exponential algorithmic speedup by a quantum walk," in *Proceedings of the Thirty-Fifth Annual ACM Symposium on Theory of Computing*, ser. STOC '03. New York, NY, USA: Association for Computing Machinery, 2003, p. 59–68. [Online]. Available: https://doi.org/10.1145/780542.780552

[5] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010.

[6] A. Javadi-Abhari, M. Treinish, K. Krsulich, C. J. Wood, J. Lishman, J. Gacon, S. Martiel, P. D. Nation, L. S. Bishop, A. W. Cross, B. R. Johnson, and J. M. Gambetta, "Quantum computing with Qiskit," 2024.

[7] N. Khammassi, I. Ashraf, J. V. Someren, R. Nane, A. M. Krol, M. A. Rol, L. Lao, K. Bertels, and C. G. Almudever, "Openql: A portable quantum programming framework for quantum accelerators," *J. Emerg. Technol. Comput. Syst.*, vol. 18, no. 1, Dec. 2021. [Online]. Available: https://doi.org/10.1145/3474222

[8] S. van der Linde, W. de Kok, T. Bontekoe, and S. Feld, "qgym: A gym for training and benchmarking rl-based quantum compilation," arXiv.org, 2023. [Online]. Available: https://arxiv.org/abs/2308.02536

[9] G. Li, Y. Ding, and Y. Xie, "Tackling the qubit mapping problem for nisq-era quantum devices," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 1001–1014. [Online]. Available: https://doi.org/10.1145/3297858.3304023

[10] A. Cowtan, S. Dilkes, R. Duncan, A. Krajenbrink, W. Simmons, and S. Sivarajah, "On the Qubit Routing Problem," in *14th Conference on the Theory of Quantum Computation, Communication and Cryptography (TQC 2019)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), W. van Dam and L. Mančinska, Eds., vol. 135. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2019, pp. 5:1–5:32. [Online]. Available: https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.TQC.2019.5

[11] M. G. Pozzi, S. J. Herbert, A. Sengupta, and R. D. Mullins, "Using reinforcement learning to perform qubit routing in quantum compilers," *ACM Transactions on Quantum Computing*, vol. 3, no. 2, pp. 1–25, 2022.

[12] M. Y. Siraichi, V. F. d. Santos, S. Collange, and F. M. Q. Pereira, "Qubit allocation," in *Proceedings of the 2018 International Symposium on Code Generation and Optimization*, ser. CGO 2018. Association for Computing Machinery, 2018, pp. 113–125.

[13] P. Murali, J. M. Baker, A. Javadi-Abhari, F. T. Chong, and M. Martonosi, "Noise-adaptive compiler mappings for noisy intermediate-scale quantum computers," in *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 1015–1029. [Online]. Available: https://doi.org/10.1145/3297858.3304075

[14] J. Preskill, "Quantum computing in the NISQ era and beyond," *Quantum*, vol. 2, p. 79, 2018. [Online]. Available: https://doi.org/10.22331/q-2018-08-06-79

[15] W. Tang, Y. Duan, Y. Kharkov, R. Fakoor, E. Kessler, and Y. Shi, "Alpharouter: Quantum circuit routing with reinforcement learning and tree search," in *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 01, 2024, pp. 930–940.

[16] S. Herbert and A. Sengupta, "Using reinforcement learning to find efficient qubit routing policies for deployment in near-term quantum computers," arXiv.org, 2018. [Online]. Available: https://arxiv.org/abs/1812.11619

[17] N. Quetschlich, L. Burgholzer, and R. Wille, "Compiler optimization for quantum computing using reinforcement learning," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6. [Online]. Available: https://doi.org/10.1109/DAC56929.2023.10248002

[18] T. Fösel, M. Y. Niu, F. Marquardt, and L. Li, "Quantum circuit optimization with deep reinforcement learning," arXiv.org, 2021. [Online]. Available: https://arxiv.org/abs/2103.07585

[19] R. A. Oancea, S. van der Linde, W. de Kok, M. Sabatelli, and S. Feld, "Optimizing initial qubit mappings under fixed gate error rates using deep reinforcement learning," in *Innovations for Community Services*, S. Zielinski, G. Eichler, C. Erfurth, and G. Fahrnberger, Eds. Cham: Springer Nature Switzerland, 2025, pp. 189–208.

[20] National Academies of Sciences, Engineering, and Medicine, *Reproducibility and Replicability in Science*. Washington, DC: The National Academies Press, 2019. [Online]. Available: https://doi.org/10.17226/25303

[21] P. Ivie and D. Thain, "Reproducibility in scientific computing," *ACM Computing Surveys*, vol. 51, no. 3, pp. 63:1–63:36, 2018. [Online]. Available: https://doi.org/10.1145/3186266

[22] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2018, pp. 3207–3214.

[23] A. Trisovic, M. K. Lau, T. Pasquier, and M. Crosas, "A large-scale study on research code quality and execution," *Scientific Data*, vol. 9, no. 60, 2022. [Online]. Available: https://doi.org/10.1038/s41597-022-01143-6

[24] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018. [Online]. Available: http://incompleteideas.net/book/the-book-2nd.html

[25] G. Brockman, V. Cheung, L. Pettersson, J. Schneider, J. Schulman, J. Tang, and W. Zaremba, "OpenAI Gym," 2016.

[26] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S. Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I. Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of go with deep neural networks and tree search," *Nature*, vol. 529, pp. 484–489, 2016.

[27] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel, T. Lillicrap, K. Simonyan, and D. Hassabis, "A general reinforcement learning algorithm that masters chess, shogi, and go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.

[28] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton, "A survey of monte carlo tree search methods," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, 2012.

[29] L. Kocsis and C. Szepesvári, "Bandit based monte-carlo planning," in *Machine Learning: ECML 2006*, ser. Lecture Notes in Computer Science, vol. 4212. Springer, 2006, pp. 282–293.

[30] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008. [Online]. Available: https://papers.nips.cc/paper/7181-attention-is-all-you-need

[31] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proceedings of the 26th Annual International Conference on Machine Learning*, ser. ICML '09. New York, NY, USA: Association for Computing Machinery, 2009, pp. 41–48.

[32] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone, "Curriculum learning for reinforcement learning domains: A framework and survey," *Journal of Machine Learning Research*, vol. 21, no. 181, pp. 1–50, 2020. [Online]. Available: https://jmlr.org/papers/v21/20-212.html

[33] Delft High Performance Computing Centre (DHPC), "DelftBlue Supercomputer (Phase 2)," https://www.tudelft.nl/dhpc/ark:/44463/DelftBluePhase2, 2024.